Retrieval Engine Guide from Zero to 100

A clearer, step-by-step study guide for the competition notebook and report

Purpose

This guide explains the retrieval project notebook from the ground up. It starts with the basic language of information retrieval, then moves to TF-IDF, BM25+, dense embeddings, category classification, cross-encoder reranking, validation splits, leakage-safe experimentation, evaluation, and performance engineering. Compared with the earlier version, this edition also adds slower explanations, symbol-by-symbol formula reading, notebook-aligned code examples, and a more complete discussion of why the pipeline is now a two-stage retrieval system rather than a single retriever with a light post-filter.

Be careful

Two common misconceptions should be corrected from the start.

- **TF-IDF is not just repeated-word counting.** It weights within-document frequency by how rare the term is across the corpus.
- **BM25+ is not mainly a tokenizer or stopword remover.** BM25+ is a probabilistic lexical ranking function with term-frequency saturation and document-length normalization. Tokenization and stopword removal are optional preprocessing choices, not the core formula.

Guide map

```
START
  |
  +--> Section 1: how to read the notebook as a system
  |
  +--> Section 2: retrieval basics from zero
  |
  +--> Section 3: preprocessing and text construction
  |
  +--> Section 4: TF-IDF in an academic but readable way
  |
  +--> Section 5: BM25+ and why it usually beats plain TF-IDF lexically
  |
  +--> Section 6: dense embeddings and semantic retrieval
  |
  +--> Section 7: two-stage retrieval, cross-encoders, and category signals
  |
  +--> Section 8: validation, leakage, and honest experimentation
  |
  +--> Section 9: evaluation metrics and score interpretation
  |
  +--> Section 10: caching, chunking, argpartition, and runtime design
  |
  +--> Section 11: notebook walkthrough and full pipeline mental model
  |
  `--> Appendix: glossary, formulas, worked code examples, and oral-exam answers
```

March 27, 2026

1 How to use this guide

Suggested reading order

Read Sections 2, 3, 4, 5, and 6 in order. After that, read Sections 7 and 8 carefully before moving on to Sections 9 and 10. That sequence matches the actual learning path: understand the task first, then the models, then why one stage is not enough, then why validation protocol matters, then the metrics, and only after that the engineering.

- **First pass:** Focus on vocabulary. Learn corpus, query, ranking, relevance, token, embedding, recall, precision, and MRR.
- **Second pass:** Focus on formulas. Understand exactly what TF-IDF, BM25+, cosine similarity, and a linear SVM are doing.
- **Third pass:** Focus on the notebook workflow. Connect each notebook block to the conceptual reason behind it.
- **Final pass:** Focus on experimentation. Only then tune `top_k`, embedding batch size, or filtering strategy.

Note

This competition pipeline combines TF-IDF and BM25+ baselines, a dense embedding retriever as the first-stage active model, a TF-IDF + LinearSVC category classifier, a cross-encoder reranker for the top candidates, and a leakage-safe validation protocol for tuning `top_k`, `top_m`, and category bonus before final submission.

2 The retrieval problem

2.1 What is information retrieval?

Information retrieval asks a simple question: given a query, which documents in a large collection should be ranked highest? In this project, the collection contains technical forum documents, and each query is a short technical problem statement. The system returns a ranked list of document identifiers.

The central objects are:

$$\mathcal{D} = \{d_1, d_2, \ldots, d_N\} \quad \text{(documents)}, \qquad \mathcal{Q} = \{q_1, q_2, \ldots, q_M\} \quad \text{(queries)}.$$

For one query q, the retriever produces an ordering

$$\pi_q = (d_{i_1}, d_{i_2}, \ldots, d_{i_K}),$$

where earlier positions are supposed to be more relevant.

2.2 What does “relevant” mean?

A document is relevant to a query if it helps answer that query according to the competition ground truth. Relevance is not identical to textual overlap. A relevant document can use different words from the query but still discuss the same underlying issue. This distinction is exactly why sparse lexical methods and dense semantic methods behave differently.

2.3 What is a ranking score?

A ranking function assigns a real number to each query-document pair:

$$\text{score}(q, d) \in \mathbb{R}.$$

The system sorts documents by descending score. Different retrieval methods define this score differently:

- TF-IDF usually uses cosine similarity between sparse vectors.
- BM25+ uses a token-level lexical relevance formula.
- Dense retrieval uses similarity between learned dense vectors.

2.4 Precision, recall, and evaluation metrics (quick recap)

This is one of the places where students often get lost, so let us slow down and separate the notation from the idea.

For one query q:

- R_q means "the truly relevant documents for this query".
- S_q^K means "the documents our system returned in the top K positions".
- $|S_q^K \cap R_q|$ means "how many of those top-K returned documents were actually relevant".

So the quantity $|S_q^K \cap R_q|$ is simply the **number of hits inside the top-K**. Once that is clear, the formulas become much less mysterious.

Read the metrics in plain English

For one query, start with this question: *how many relevant documents did we successfully place in the top K?* Call that number the **hits**. Then:

- **Precision@K** = hits divided by K. It asks: among the K documents we returned, what fraction was correct?
- **Recall@K** = hits divided by the total number of truly relevant documents. It asks: among everything relevant that existed, what fraction did we recover?
- **F1@K** combines precision and recall into one number when you want a balance between both.

Formally, with

$$S_q^K = \{d_{i_1}, \ldots, d_{i_K}\},$$

we have

$$\text{Precision@}K = \frac{|S_q^K \cap R_q|}{K} \qquad \text{and} \qquad \text{Recall@}K = \frac{|S_q^K \cap R_q|}{|R_q|}.$$

The F1 score at cutoff K is

$$\text{F1@}K = \frac{2\,\text{Precision@}K\,\text{Recall@}K}{\text{Precision@}K + \text{Recall@}K}.$$

A concrete mini-example

Suppose one query has exactly three relevant documents:

$$R_q = \{d_3, d_8, d_{20}\}.$$

Now suppose the system returns the following top-5 list:

$$S_q^5 = \{d_8, d_2, d_3, d_{11}, d_{40}\}.$$

Then the relevant hits inside the top-5 are d_8 and d_3, so

$$|S_q^5 \cap R_q| = 2.$$

Therefore:

$$\text{Precision@5} = \frac{2}{5} = 0.40, \qquad \text{Recall@5} = \frac{2}{3} \approx 0.67.$$

Interpretation: the system found two relevant documents, which is good for recall, but three of the returned five were not relevant, which lowers precision.

Two key intuitions follow immediately:

- Increasing K usually **raises recall**, because you keep more candidates and are less likely to miss relevant items.
- Increasing K often **lowers precision**, because the denominator K grows and later ranks usually contain more noise.

Another common ranking metric is **MRR** (Mean Reciprocal Rank). MRR does *not* ask how many relevant documents you found. It asks how early the *first* relevant document appeared.

For one query, if the first relevant document appears at rank r_q, then the reciprocal rank is

$$\text{RR}(q) = \frac{1}{r_q}.$$

Across all queries,

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \text{RR}(q) = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{r_q}.$$

If the first relevant result is at rank 1, the reciprocal rank is 1. If it is at rank 5, the reciprocal rank is $1/5 = 0.2$. So high MRR means relevant documents show up early, not merely somewhere far down the ranking.

All of these metrics are computed query by query and then averaged across queries.

2.5 What is top-k?

`top_k` is the number of documents retained for each query after the first-stage retriever. Small K can miss relevant documents. Large K improves recall but usually hurts precision and increases compute and output size.

One-query view of the pipeline

```
query q
  |
  v
[text normalization]
  |
  v
[first-stage retriever]
  |------------------------------|
  | TF-IDF     BM25+   Embedding |
  |------------------------------|
  |
  v
ranked list of top_k documents
  |
  +--> optional cross-encoder reranking of top_m
  |
  `--> optional category-aware score bonus
  |
  v
final ranked submission / offline evaluation
```

2.6 Why lexical and semantic retrieval differ

Lexical retrieval is driven by token overlap. It is strong when exact words matter, such as error messages, API names, product names, flags, and codes. Semantic retrieval tries to map different phrasings with similar meaning to nearby vectors. It is stronger when the query paraphrases the document instead of repeating the same words.

Warning

A query and a relevant document can talk about the same issue while sharing few exact words. That failure mode is often called **vocabulary mismatch** or **semantic mismatch**. This is the main reason embedding retrieval became the final active method in the notebook.

3 Dataset, text construction, and preprocessing

3.1 Dataset objects in the notebook

The project materials describe a collection of 216,041 documents, 327 training queries, 141 test queries, and a training relevance file. Documents belong to five categories: `tex`, `unix`, `gaming`, `programmers`, and `android`. That category signal later becomes useful for classification and filtering.

3.2 Why strict schema checks matter

The notebook performs early validation of columns and identifier uniqueness. This is not cosmetic. Retrieval systems break quietly when ids are duplicated, when expected fields are missing, or when train and test schemas differ slightly.

Conceptually, early validation prevents these classes of bugs:

- a document missing its text field,
- duplicate ids that later corrupt ranking output,
- category labels absent during classifier training,
- broken merges between results and relevance judgments.

3.3 How text is constructed

The notebook builds retrieval documents from

`title + text + tags`,

while retrieval queries use

`title + text`.

For the classifier, the notebook can use query tags as well. This distinction matters because the retriever and classifier are not solving exactly the same subproblem.

Why combine multiple fields?

A forum post title is often concise and highly informative, the body provides detailed context, and tags provide compact topical labels. Merging them can increase retrieval signal. But you should merge them consistently across train and test to keep the feature space stable.

3.4 Normalization policy

The notebook uses light, shared normalization instead of aggressive linguistic processing. In simplified form, the policy is:

1. replace separators `-`, `\_`, and `/` with spaces,
2. lowercase the text,
3. collapse repeated whitespace,
4. strip leading and trailing space.

This produces consistent text while preserving most lexical information.

Listing 1: Conceptual normalization logic

```
def normalize_text(text):
    text = replace_separators(text, chars='-_/')
    text = text.lower()
    text = collapse_whitespace(text)
    return text.strip()
```

3.5 Tokenization in this notebook

The lexical token pattern is

`[a-z0-9]+`.

That means tokens are sequences of lowercase letters and digits. So numbers are not automatically removed; they are kept whenever they match the pattern.

Warning

In the current notebook, stopword filtering is **enabled for lexical methods** (BM25+, TF-IDF, classifier TF-IDF) through the tokenizer and vectorizer settings. Words such as `the`, `is`, `are`, and `or` are removed from lexical features by default. You can disable this behavior via `STOPWORD_FILTER_ENABLED=False`.

3.6 Why use shared preprocessing across all retrievers?

If TF-IDF, BM25+, and embeddings receive differently normalized text, comparison becomes noisy. You no longer know whether a performance change came from the model or from preprocessing differences. Shared normalization is therefore an experimental control.

4 TF-IDF from zero

4.1 The bag-of-words idea

TF-IDF starts from a sparse representation. Imagine a vocabulary

$$V = \{t_1, t_2, \ldots, t_{|V|}\}.$$

Each document becomes a vector of size $|V|$. Most entries are zero because each document only uses a small subset of all possible words. This is why TF-IDF is called a **sparse** representation.

4.2 Term frequency

For a term t and document d, term frequency $\mathrm{tf}(t, d)$ measures how often t appears in d. A larger count usually suggests the term is more central to that document.

Example: if the word `kernel` appears 6 times in document d_1 but only once in document d_2, then $\mathrm{tf}(\texttt{kernel}, d_1)$ is larger than $\mathrm{tf}(\texttt{kernel}, d_2)$. Inside d_1, the term looks more important.

But raw repetition alone is not enough. If a word occurs in almost every document, it is not very helpful for distinguishing relevant from irrelevant documents.

4.3 Inverse document frequency

Let N be the total number of documents and $\mathrm{df}(t)$ the number of documents containing term t. Inverse document frequency reduces the importance of overly common terms and boosts rarer terms. A standard form is:

$$\mathrm{idf}(t) = \log\left(\frac{N}{\mathrm{df}(t)}\right).$$

Different libraries use smoothed variants, but the intuition is always the same: rare terms carry more discriminative information.

4.4 Putting TF and IDF together

A simple TF-IDF weight is

$$\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \text{idf}(t).$$

So TF-IDF is not merely counting repeated words. It answers a more refined question:

> Is this term frequent in this document *and* relatively rare in the corpus?

A tiny TF-IDF example

Imagine a corpus with three documents:

- d_1: `kernelpanicafterupdate`
- d_2: `wifidriverupdateissue`
- d_3: `kernelmodulecompilation`
  Now compare the terms `kernel` and `panic`.
- `kernel` appears in d_1 and d_3, so its document frequency is 2.
- `panic` appears only in d_1, so its document frequency is 1.

That means `panic` gets a larger IDF than `kernel`, because it is rarer. If the query is `kernelpanic`, document d_1 receives the strongest signal because it contains *both* terms, including the rarer and more discriminative one.

4.5 Query and document vectors

The query is also turned into a TF-IDF vector. Then the model computes similarity between the sparse query vector and each sparse document vector.

The most common similarity is cosine similarity:

$$\cos(\theta) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \, \|\mathbf{d}\|}.$$

This measures angle rather than raw length, which makes the comparison more stable when queries and documents have different sizes.

Tip

A practical way to read cosine similarity is this: **the numerator $\mathbf{q} \cdot \mathbf{d}$** rewards shared weighted terms, while **the denominator** rescales by vector length so very long documents do not automatically dominate just because they contain more total words.

TF-IDF intuition

```
rare term in query + rare term in document  --> strong signal
common term in query + common term in document --> weak signal
no exact overlap --> almost no lexical signal
```

4.6 Why the notebook uses unigrams and bigrams

The notebook uses `ngram_range=(1,2)`. That means it includes:

- unigrams: single tokens such as `kernel`
- bigrams: adjacent token pairs such as `kernelpanic`

Bigrams help preserve short phrases and can improve lexical specificity.

The notebook also uses `min_df=2`, meaning terms that appear in only one document are pruned. This reduces very rare noise, though the vectorizer can fall back to `min_df=1` if pruning removes everything in an edge case.

4.7 Strengths of TF-IDF

- simple and fast,
- easy to inspect,
- strong on exact technical terms,
- good as a baseline and debugging instrument.

4.8 Weaknesses of TF-IDF

- weak on paraphrases,
- weak when query and document use synonyms,
- less robust when documents are very long and heterogeneous,
- still largely dependent on direct overlap.

Tip

In oral or written explanation, a clean sentence is: "TF-IDF is a sparse lexical weighting scheme that emphasizes terms that are frequent within a document but rare across the collection, and it is typically paired with cosine similarity for ranking."

5 BM25+ from zero

5.1 Why BM25 exists if we already have TF-IDF

TF-IDF is a good baseline, but it does not model two practical effects very elegantly:

1. repeated occurrences of a term should help, but with diminishing returns,
2. long documents should not win only because they contain more words.

BM25 was designed to address these issues more directly.

5.2 The core BM25 idea

For each query term t, BM25 computes a contribution to the score of document d. A standard BM25+ form is

$$\text{score}(q,d) = \sum_{t \in q} \text{idf}(t) \left(\frac{f(t,d)(k_1+1)}{f(t,d) + k_1 \left(1 - b + b\frac{|d|}{\text{avgdl}}\right)} + \delta \right).$$

Do not try to memorize that whole formula at once. Read it in layers:

1. **Loop over the query terms.** The outer sum says: for every term in the query, add that term's contribution.
2. **Give rare terms more power.** The factor $\text{idf}(t)$ says rarer terms matter more.
3. **Reward term repetition, but not linearly forever.** The fraction involving $f(t,d)$ increases when the term appears more often, but the gain gradually saturates.
4. **Correct for document length.** The $|d|/\text{avgdl}$ part prevents long documents from winning too easily just because they contain many words.
5. **Add a small positive offset.** The δ term is the BM25+ refinement.

The symbols mean:

- $f(t,d)$: how many times term t appears in document d,
- $|d|$: document length,
- avgdl: average document length in the corpus,
- k_1: how quickly the term-frequency reward saturates,
- b: how strongly length normalization is applied,
- δ: the BM25+ offset.

5.3 Term-frequency saturation

BM25 does not reward repeated terms linearly forever. The first few appearances matter more than the fiftieth. This is called **term-frequency saturation**. It reflects the intuition that after a point, additional repetitions add less new evidence.

A simple intuition is this: seeing `kernel` once already tells you the document is about the kernel. Seeing it 40 times does strengthen that belief, but not by a factor of 40. BM25 models that diminishing return explicitly.

5.4 Length normalization

If two documents mention the same important term equally well, a much longer document should not automatically be preferred just because it contains more total words. The b parameter adjusts for this by comparing document length to average document length.

For example, if document A has 120 words and document B has 2200 words, and both contain the same important query term a similar number of times, BM25 usually avoids over-rewarding the very long document. That is one reason BM25 often behaves more naturally than naive counting.

5.5 What BM25+ adds

BM25+ adds the δ term so that long documents are not overly penalized when they do contain a query term. It is a refinement of BM25, especially helpful in some long-document settings.

A two-document intuition check

Suppose the query is `kernelpanic`.

- Document A is short, focused, and contains both terms clearly.
- Document B is much longer, contains `kernel` many times, but mostly as background discussion.

TF-IDF may already prefer A, but BM25+ is often even better at this kind of situation because it explicitly controls both repetition and document length. In plain language, BM25+ says: *repetition helps, but only up to a point, and long documents should be treated carefully.*

5.6 How to interpret the notebook parameters

The notebook uses:

$$k_1 = 1.5, \qquad b = 0.75, \qquad \delta = 1.0.$$

These are reasonable, standard-looking defaults:

- $k_1 = 1.5$: moderate saturation,
- $b = 0.75$: substantial but not extreme length normalization,
- $\delta = 1.0$: positive BM25+ adjustment.

5.7 Correcting the stopword misconception

Your intuition that BM25 cares about common words is directionally related to reality, but the mechanism is different from "BM25 removes the, a, is".

The correct explanation is:

- BM25 is still a lexical matching model based on tokens.
- Common terms often receive lower idf, so they contribute less to ranking.
- Tokenization and stopword removal happen *before* scoring if the pipeline designer chooses them.
- In this notebook, the tokenizer keeps lowercase alphanumeric tokens, including numbers, and applies configurable stopword filtering for lexical retrieval.

5.8 Why BM25+ can beat TF-IDF lexically

On long and uneven documents, BM25+ often gives more realistic lexical scores because it combines term rarity, saturation, and length normalization in a more retrieval-specific way than plain TF-IDF cosine similarity.

BM25+ intuition in one line

```
TF-IDF asks: which rare terms overlap?
BM25+ asks: which rare terms overlap, with sensible diminishing returns and document-
    length correction?
```

5.9 What BM25+ still cannot solve

BM25+ is still lexical. If a relevant document says `applicationfreezes` and the query says `programhangs`, BM25+ may still miss the connection if the shared token overlap is weak.

6 Dense embeddings and semantic retrieval

6.1 Why move from sparse to dense representations?

Sparse vectors have one dimension per vocabulary item. Dense vectors instead have a fixed low dimension and are learned so that semantically similar texts land close to one another in vector space.

In the project notebook, the active dense model is `all-MiniLM-L6-v2`. The report describes document and query vectors of dimension 384.

6.2 What an embedding is

An embedding is a dense vector representation:

$$\mathbf{e}(x) \in \mathbb{R}^{384}.$$

The important idea is not the exact dimension; it is that semantic information is distributed across the vector coordinates.

In other words, the vector is not "count word 1, count word 2, count word 3". Instead, it is a learned numerical representation whose coordinates only make sense together. One coordinate by itself usually has no simple human interpretation; the meaning lives in the full pattern.

6.3 How dense retrieval works

The notebook encodes every document into a dense vector and stores the resulting matrix. Each query is also encoded into a dense vector. Then the retriever computes similarity between the query vector and all document vectors.

If vectors are L2-normalized, cosine similarity and dot product become equivalent up to scale:

$$\cos(\theta) = \mathbf{e}(q)^\top \mathbf{e}(d) \quad \text{when both vectors have unit norm.}$$

A semantic example

Suppose the query says "program hangs during launch" and a relevant document says "application freezes on startup". A lexical retriever may struggle because `hangs` is not the same token as `freezes`, and `launch` is not the same token as `startup`. A dense embedding model can still place those two texts near each other because it learns semantic similarity rather than relying only on exact token overlap.

6.4 Why dense retrieval helps here

Dense retrieval is especially strong when:

- the query paraphrases the relevant document,
- the same issue is described with different wording,
- abbreviations and alternate phrasing appear,
- exact lexical overlap is sparse.

Semantic shift

The notebook moves from lexical matching to semantic matching. Instead of asking only "which tokens overlap?", the dense model asks "which texts are close in learned meaning space?".

6.5 Why embeddings are expensive

Dense retrieval usually improves recall, but it introduces computational cost:

- encoding all documents is expensive,
- storing the embedding matrix takes memory,
- scoring query vectors against all document vectors can be slow.

This is why the notebook invests heavily in caching and chunked scoring.

6.6 Batching and chunking

The notebook uses an embedding batch size for encoding and a query chunk size for scoring. These are engineering controls, not retrieval theory. They do not change the mathematical objective; they change runtime and memory behavior.

If you score all queries against all documents at once, you may create a huge dense score matrix. Chunking processes manageable query groups instead.

6.7 What dense retrieval does not magically solve

Dense retrieval is not perfect.

- It can be slower.
- It can require more memory.
- It can produce semantically plausible but category-wrong documents.
- It may need post-filtering or reranking for best precision.

Warning

Dense retrieval is usually a recall tool first. In many systems, it brings more relevant candidates into the top-K, but later stages are still needed to clean noise.

7 Two-stage retrieval, cross-encoders, and category signals

7.1 Why a single retriever is often not enough

Students often imagine retrieval as one scoring formula applied once. In practice, high-performing systems are frequently **multi-stage**. One component tries to avoid missing relevant documents, while a later component tries to sort the most promising candidates more precisely.

That is exactly the logic of the current notebook:

1. a **first-stage retriever** retrieves a large candidate set with high recall,
2. a **cross-encoder reranker** looks more carefully at the top part of that candidate set,
3. a **category signal** nudges the reranked scores so category-consistent documents receive a small bonus instead of a hard cut.

Two jobs, not one

The first stage asks: *which 7,500 documents are worth keeping at all?* That is mostly a recall problem. The second stage asks: *inside the most promising 20, 30, 50, or 100 candidates, which ones deserve the very top ranks?* That is mostly a precision and ordering problem.

7.2 Candidate generation versus reranking

The first-stage embedding retriever is built to be efficient at large scale. It encodes documents once, encodes each query, and computes many similarities quickly. That makes it practical to search hundreds of thousands of documents.

But this speed comes with a limitation: the query and the document are encoded *independently*. This is often called a **bi-encoder** pattern. It is excellent for retrieval speed, but it is not the most detailed way to judge whether one specific query-document pair is truly relevant.

The cross-encoder solves that later, but only for a much smaller set of candidates.

Two-stage retrieval in one query

```
query q
  |
  v
first-stage retriever
  |
  +--> keep many candidates (large top_k)
  |
  v
top_k candidate list
  |
  v
cross-encoder rerank of only top_m candidates
  |
  v
soft category bonus added to reranked scores
  |
  v
final ranked list
```

7.3 What a cross-encoder is, from zero

A cross-encoder is a model that receives the *query and document together* and produces one score for the pair. In notation, we can write:

$$\text{score}_{\text{cross}}(q, d) = f_\theta(q, d),$$

where f_θ is a learned neural network.

A useful contrast is:

- **Embedding retriever / bi-encoder:** encode q once, encode d once, compare the vectors.
- **Cross-encoder:** feed the exact pair (q, d) into the model and ask for one joint relevance score.

This joint processing is more expressive because the model can directly reason about interactions between query words and document words. For example, it can notice that `startup` in the document relates to `launch` in the query *in this specific pair*, rather than relying only on a global vector similarity.

7.4 Why a cross-encoder is more accurate but slower

A cross-encoder is usually more accurate at fine-grained ranking because it sees both texts at the same time. But it is dramatically slower at large scale, because it cannot precompute one fixed document vector and reuse it for every query. Instead, it must evaluate many query-document pairs individually.

That is why the notebook does *not* run the cross-encoder on all 216,041 documents for every query. It would be far too expensive. Instead, it uses the fast embedding stage to narrow the search space first.

Think of the cross-encoder as a careful judge

The embedding retriever is a fast scout. It quickly finds many plausible candidates. The cross-encoder is a slower judge. It cannot inspect the whole collection, but it can make better decisions when you show it a shortlist.

7.5 Why rerank only the top-m candidates?

Suppose `top_k=7500`. That large candidate pool is useful for recall, but reranking all 7,500 candidates with a cross-encoder for every query would still be expensive. The notebook therefore uses another cutoff, `top_m`, which might be 20, 30, 50, or 100.

The logic is:

1. let the first-stage retriever find many potentially relevant documents,
2. assume the most important ordering mistakes happen near the top,
3. spend the expensive cross-encoder budget only where it matters most.

This is why `top_k` and `top_m` solve different problems:

- `top_k`: how many candidates do we keep after the fast retrieval stage?
- `top_m`: how many of those candidates do we rescore with the expensive reranker?

7.6 How the notebook trains the cross-encoder

The current notebook trains the cross-encoder with binary query-document pairs:

- **positive pairs:** a train query and one of its relevant documents,
- **negative pairs:** a train query and a document that is not relevant.

The model is therefore not learning "category" directly. It is learning a more local question:

> Given this exact query and this exact document, should this pair receive a high relevance score or a low one?

7.7 What positives, negatives, and hard negatives mean

Positive examples are easy to understand: they come from the ground-truth relevant documents.

Negative examples need more care. There are two broad kinds:

- **random negatives:** documents sampled from the corpus that are assumed not to be relevant,
- **hard negatives:** documents that the first-stage retriever ranked highly but that are not actually relevant.

Hard negatives are especially useful because they are the mistakes the model is most likely to make. They are often semantically similar yet still wrong, so they teach the reranker a more difficult discrimination task.

Why hard negatives matter

Imagine the query is `androidappcrashesonstartup`. A random negative might be an unrelated `tex` document about LaTeX footnotes. That negative is too easy. A hard negative is more like a `programmers` or `android` document about `appfreezeswhenloadingimages`. That pair looks much more plausible, so learning to reject it is more valuable.

7.8 A notebook-aligned code sketch of cross-encoder training

Listing 2: Simplified logic for building cross-encoder training pairs

```
query_text_map = build_text_map(train_queries_frame)
doc_text_map = build_text_map(docs_frame)

hard_negative_doc_ids_by_query = mine_hard_negative_doc_ids(
    train_queries_frame=train_queries_frame,
    docs_frame=docs_frame,
    ground_truth=ground_truth,
    query_ids=candidate_query_ids,
    top_k=CROSS_ENCODER_HARD_NEGATIVE_TOP_K,
)

training_examples = build_cross_encoder_training_examples(
    query_text_map=query_text_map,
    doc_text_map=doc_text_map,
    ground_truth=ground_truth,
    query_ids=candidate_query_ids,
    max_positives_per_query=4,
    negatives_per_positive=4,
    hard_negative_doc_ids_by_query=hard_negative_doc_ids_by_query,
)

cross_encoder.fit(train_dataloader=train_loader, epochs=5)
```

Read that code in words:

1. build query and document text lookups,
2. mine difficult wrong documents using the first-stage retriever,
3. assemble positive and negative training pairs,
4. fit the cross-encoder on those pairs.

7.9 What the category classifier is doing here

The retriever and reranker score relevance. The classifier predicts a category label for each query. These are different tasks.

In this notebook, the category classifier is trained with TF-IDF features and a linear support vector machine:

- features: TF-IDF vectors,
- classifier: `LinearSVC`,
- label: one category per query.

7.10 What a linear SVM is doing here

A linear SVM learns a separating hyperplane between classes in feature space. With a multiclass scheme, it predicts one category for each query from its TF-IDF representation.

A clean academic description is:

> The classifier is a linear discriminative model over sparse TF-IDF features, trained to predict the most likely forum category of each query.

7.11 How the category bonus works

Earlier versions of the project emphasized a strict dominant-category filter. The current notebook uses a softer idea during reranking: if the predicted query category matches the document category, the system adds a small scalar bonus to the reranked score.

In simplified form:

$$\text{final\_score}(q, d) = \text{cross\_score}(q, d) + \text{bonus}(q, d),$$

where

$$\text{bonus}(q, d) = \begin{cases} \lambda & \text{if predicted query category} = \text{document category} \\ 0 & \text{otherwise.} \end{cases}$$

So the category model does *not* decide relevance on its own. It only nudges the cross-encoder ranking a little.

Warning

This soft bonus is much safer than a hard filter. A hard filter can delete relevant documents entirely if the predicted category is wrong. A soft bonus only shifts scores; it does not force all nonmatching documents to disappear.

7.12 The older dominant-category filter versus the newer soft bonus

The guide still discusses the dominant-category filter because it is educational and still implemented as a utility. But conceptually, the two strategies are different:

- **dominant-category filter:** choose one category and remove documents outside it,
- **soft category bonus:** keep all documents but slightly reward category agreement.

The soft bonus is the more conservative and generally more robust choice.

7.13 A complete toy reranking example

Suppose the embedding retriever returns these five candidates for one query:

Document	Embedding rank	Cross-encoder score	Category match bonus
d_1	1	1.20	+0.0
d_2	2	2.05	+0.5
d_3	3	1.95	+0.0
d_4	4	1.30	+0.5
d_5	5	0.80	+0.5

Then the final reranking scores become:

$$d_1 : 1.20, \quad d_2 : 2.55, \quad d_3 : 1.95, \quad d_4 : 1.80, \quad d_5 : 1.30.$$

So the final order becomes

$$d_2 \succ d_3 \succ d_4 \succ d_5 \succ d_1.$$

Notice what happened:

- the embedding stage decided which five candidates deserved attention,
- the cross-encoder changed the local ordering,
- the category bonus gave an extra nudge but did not dominate the whole score.

7.14 What to say in one sentence

If you need a short explanation for class, say this:

> The notebook uses a two-stage retrieval design: a fast dense bi-encoder retrieves a large candidate pool, then a cross-encoder reranks only the top candidates more accurately, with a lightweight category bonus added as a soft prior.

8 Validation, leakage, and honest experimentation

8.1 Train, validation, and test from zero

These words are used constantly in machine learning, but students are often not told slowly enough what they mean.

- **Train set:** data used to fit model parameters.
- **Validation set:** data used to choose hyperparameters and compare candidate settings.
- **Test set:** data reserved for the final unbiased evaluation or submission.

In plain language:

- the model *learns* from the train set,
- the researcher *decides* settings using the validation set,
- the final system is *judged* on the test set.

8.2 What a hyperparameter is

A hyperparameter is a setting chosen by the experimenter rather than learned directly by gradient descent or fitting. In this notebook, examples include:

- `top_k`,
- `top_m`,
- category bonus strength,
- number of cross-encoder epochs,
- number of negatives per positive pair.

You do not want to choose these by looking at the same examples that the model already learned from, because then you start adapting your whole system to that same data repeatedly.

8.3 What leakage means

Data leakage means information from the evaluation target leaks into training or tuning in a way that makes the measured score look better than it really is.

Leakage does not always mean a programming bug. Sometimes it is an experimental-design bug. A very common form is:

> train a model on one set of labeled examples, then evaluate it on those same examples and treat the result as if it were a fair estimate of generalization.

8.4 Why evaluating on the same 327 train queries was optimistic

Suppose the cross-encoder is trained on all 327 labeled queries. Then the offline evaluation loop also runs on those same 327 queries. Even if the relevant documents are numerous and the pair construction is indirect, the system has still been tuned and fit on the very queries used for scoring. That is optimistic.

Why?

- the cross-encoder has already seen those queries during fitting,
- the hard negatives are mined from those same labeled queries,
- the researcher may also choose `top_k`, `top_m`, and bonus strength based on those same results.

At that point, the offline score is not a clean estimate of how well the system will behave on unseen queries. It is partly a measurement of how well the system adapted to the known training queries.

Warning

An evaluation score is only honest if the examples used to choose the system are separated from the examples used to judge the system.

8.5 Why category-stratified splitting helps

The 327 labeled queries are not evenly distributed by category. The counts are:

Category	Labeled queries	Approx. validation size at 20%
android	57	11
gaming	41	8
programmers	52	10
tex	130	26
unix	47	9

If you make a random split without thinking about category balance, you might accidentally place too few queries from one category in validation. Then the validation score becomes less representative. Stratification fixes that by preserving category proportions approximately.

8.6 The corrected protocol in plain English

The notebook now follows this logic:

1. split the 327 labeled queries into **cross-encoder train** and **validation**,
2. train the cross-encoder only on the train split,
3. run first-stage retrieval on validation queries,

4. tune `top_k`, `top_m`, and category bonus on validation,
5. pick the best validation configuration,
6. retrain the cross-encoder on all 327 labeled queries,
7. generate the final Kaggle submission on the test queries.

Leakage-safe protocol

```
327 labeled train queries
  |
  +--> split-train queries ----> fit cross-encoder
  |
  `--> validation queries ----> tune top_k / top_m / category bonus

best validation configuration
  |
  v
retrain cross-encoder on all 327 labeled queries
  |
  v
run final test submission
```

8.7 Why retrain on all 327 after tuning?

Once hyperparameters are chosen, there is no longer a reason to waste labeled training data. At that point, the honest model-selection step is already finished. So the notebook retrains the final cross-encoder on all available labeled train queries before producing the submission.

This is a standard pattern:

1. use validation to decide *how* to build the system,
2. then use all available training labels to build the strongest final version of that chosen system.

8.8 What numbers you should and should not report

During development, the key score is the **validation score**. That is the score used to compare settings fairly.

After final retraining on all 327 queries, you should *not* compute a new "offline score" on those same 327 queries and present it as if it were equally fair. That number would again be optimistic.

So a careful report distinguishes:

- **validation performance:** used for model selection,
- **final submission output:** generated after retraining on all labeled data,
- **public or private leaderboard score:** obtained externally on hidden test labels.

8.9 What is the difference between model parameters and protocol parameters?

Students often mix together three different things:

- **model parameters:** the learned weights inside the cross-encoder,

- **hyperparameters:** settings like `top_k`, `top_m`, and category bonus,
- **protocol choices:** train/validation split strategy, stratification, and when retraining happens.

The protocol is not a cosmetic detail. It determines whether the score means anything trustworthy.

8.10 A notebook-aligned code sketch of the validation protocol

Listing 3: Simplified leakage-safe tuning loop

```
ce_train_query_ids, val_query_ids = stratified_query_train_validation_split(
    train_queries_df,
    validation_fraction=0.2,
    seed=42,
)

ce_train_queries_df = select_rows_by_ids(train_queries_df, ce_train_query_ids)
val_queries_df = select_rows_by_ids(train_queries_df, val_query_ids)

ce_train_ground_truth = subset_ground_truth(ground_truth, ce_train_query_ids)
val_ground_truth = subset_ground_truth(ground_truth, val_query_ids)

validation_cross_encoder = build_or_load_cross_encoder(
    train_queries_frame=ce_train_queries_df,
    docs_frame=docs_df,
    ground_truth=ce_train_ground_truth,
)

for top_k in TUNING_TOP_KS:
    for top_m in TUNING_RERANK_TOP_MS:
        for bonus in TUNING_CATEGORY_BONUSES:
            results = run_retrieval(... queries_frame=val_queries_df, top_k=max(TUNING_TOP_KS))
            reranked = rerank_results_with_cross_encoder(... rerank_top_m=top_m, category_bonus=
    bonus)
            metrics = leaderboard_score(reranked, val_ground_truth, k=top_k)
```

This is one of the most important conceptual updates in the whole project. The guide is emphasizing it because good retrieval work is not only about good models; it is also about honest experimental design.

9 Evaluation metrics and how to read them

9.1 Recall@K

Recall asks: among all truly relevant documents, how many did we recover?

$$\text{Recall@K} = \frac{|R_q \cap \hat{R}_{q,K}|}{|R_q|},$$

where R_q is the set of truly relevant documents and $\hat{R}_{q,K}$ is the predicted top-K set.

High recall means the system is good at not missing relevant material.

9.2 Precision@K

Precision asks: among the documents we returned, how many are truly relevant?

$$\text{Precision@K} = \frac{|R_q \cap \hat{R}_{q,K}|}{|\hat{R}_{q,K}|}.$$

If K is very large, precision often becomes small, because the denominator grows quickly.

9.3 MRR@K

Mean Reciprocal Rank focuses on the first relevant hit. For one query, if the first relevant document appears at rank r_q, then the reciprocal rank is

$$\frac{1}{r_q}.$$

Averaging across queries gives MRR. High MRR means useful documents appear early.

9.4 Accuracy

In this project, accuracy refers to the category classifier. It asks how often the predicted category matches the true category.

9.5 The combined score

The report averages four quantities:

$$\text{Score} = \frac{1}{4}(\text{Recall} + \text{Precision} + \text{MRR} + \text{Accuracy}).$$

This means the system is not evaluated only as a retriever; it is evaluated as a retrieval-plus-category pipeline.

9.6 How to interpret a very low precision with high recall

Because the active configuration uses a huge initial $K = 7500$, precision is expected to be numerically small. That does not automatically mean the system is poor. It may simply reflect the fact that you intentionally retained a large candidate set to protect recall.

The more meaningful interpretation is joint:

- high recall says relevant documents are usually present,
- reasonable MRR says useful documents appear early,
- classifier accuracy and filtering help control category noise,
- very low precision is partly a mathematical consequence of a large output list.

10 Performance engineering: why the notebook does not stop at theory

10.1 Why engineering matters here

A theoretically good retriever is not enough if every iteration takes too long. The report notes that the embedding pipeline initially approached a severe runtime bottleneck, so the notebook adds several engineering optimizations.

10.2 Disk caching

Dense document embeddings are expensive to recompute. The notebook fingerprints relevant dataframes and configuration, then saves reusable artifacts such as embedding matrices and classical indexes to disk.

This is important because the expensive step is often document encoding, not query encoding. Once document embeddings are cached, repeated runs become dramatically cheaper.

10.3 In-memory caching

The notebook also keeps objects in memory dictionaries during a live session. This avoids unnecessary repeated loads from disk after the first use.

10.4 Hardware device selection

Modern notebook code often contains references to `cpu`, `cuda`, or `mps`. These are not retrieval methods. They are hardware execution choices:

- `cpu`: run on the processor,
- `cuda`: run on an NVIDIA GPU,
- `mps`: run on Apple Silicon through Metal.

In the retrieval notebook, dense models and cross-encoders can benefit from hardware acceleration, but stability matters too. That is why device selection is an engineering concern rather than a retrieval-theory concern. A model can be conceptually correct while still failing at runtime if the chosen hardware backend is unstable.

Tip

When you see device-selection code in the notebook, read it as *systems engineering for reliable model execution*, not as part of the ranking algorithm itself.

10.5 Chunked query scoring

Instead of scoring all queries against all documents in one giant matrix multiplication, the notebook scores query chunks, for example 32 at a time. This controls peak memory usage.

10.6 Partial sorting with np.argpartition

A full sort of all document scores for every query is expensive when you only need the best K items. `np.argpartition` is a partial-selection method. It finds the top part more cheaply than a full exact sort of the entire array. You can then sort only that smaller candidate slice.

Why partial sorting is faster

```
Need top 7,500 docs out of 216,041?

Full sort:
  sort all 216,041 scores

Partial selection:
  find only the best 7,500 region first
  then sort that region
```

10.7 Reusing one max-K ranking

During offline sweeps, if you compute a correct sorted top-7,500 ranking, then top-100, top-500, or top-1,000 prefixes can be obtained by truncation. The notebook reuses that fact instead of recomputing rankings for every smaller K.

10.8 The engineering lesson

The notebook is not only a modeling notebook. It is an engineering report. Its final quality comes from both retrieval choices and systems design.

11 Notebook-aligned walkthrough

11.1 How the notebook is organized

The saved notebook is organized in a clean progression:

1. project overview,
2. explanation of why embeddings are the default,
3. `top_k` discussion,
4. method comparison,
5. imports and configuration,
6. data loading,
7. preprocessing,
8. index construction and caching,
9. retrieval functions,
10. evaluation logic,
11. cross-encoder training utilities,
12. stratified train/validation split and tuning loop,
13. final retrain on all 327 labeled queries,
14. test submission generation,
15. conclusion.

11.2 What each code region is conceptually doing

Notebook region	Conceptual role
Imports and config	Defines global settings: active model, `top_k` candidates, `top_m` reranking depth, tokenizer pattern, TF-IDF parameters, BM25+ parameters, embedding model name, cache directories, and device selection.
Data loading	Reads raw JSON files, validates required columns, checks ids, and enforces reproducibility assumptions.
Preprocessing	Normalizes text and builds unified `content` fields for documents and queries.
Index construction	Fits TF-IDF, builds BM25+, loads the sentence-transformer, and prepares or loads cached embeddings.
Retrieval functions	Converts one method choice into a ranked list of documents for each query.
Cross-encoder utilities	Builds pairwise training examples, mines hard negatives, trains or loads the reranker, and applies reranking plus category bonus.
Evaluation logic	Computes Recall, Precision, MRR, Accuracy, and their average score on the validation split.
Experiment loop	Runs the selected models, sweeps `top_k`, `top_m`, and category bonus, and stores validation summaries.
Submission generation	Retrains the cross-encoder on all 327 labeled queries, runs the final model on the test queries, and writes the output CSV.

11.3 What is "active" and what is "implemented"

A crucial distinction in the project report is this:

- TF-IDF retrieval is implemented, but not the final active submission method.
- BM25+ retrieval is implemented, but not the final active submission method.
- Embedding retrieval is the active first-stage retriever.
- The cross-encoder is the active second-stage reranker.
- The category classifier is an active support component.
- The strict dominant-category filter is still educational and implemented, but the active notebook protocol now favors a soft category bonus during reranking.

That is an academically strong experimental structure: keep simpler baselines for comparison, but submit the method that actually wins on the observed task.

12 A complete mental model of the full pipeline

End-to-end mental model

```
1. Load documents, train queries, test queries, qrels, and submission template.
2. Validate schemas and ids.
3. Normalize text consistently.
4. Build document content and query content.
5. Train the query category classifier (TF-IDF + LinearSVC).
6. Split the 327 labeled train queries into train and validation folds.
7. Train the cross-encoder only on the split-train queries.
8. Build retrieval artifacts for TF-IDF, BM25+, and/or embeddings.
9. Run first-stage retrieval on the validation queries.
10. Keep a large top_k candidate list to protect recall.
11. Rerank only the top_m candidates with the cross-encoder.
12. Add a soft category bonus when query and document categories agree.
13. Score the validation fold and choose the best hyperparameters.
14. Retrain the cross-encoder on all 327 labeled queries.
15. Run the final model on test queries and export CSV.
```

One-sentence summary

The final system is a dense semantic first-stage retriever, followed by a cross-encoder reranker and a lightweight category-aware score bonus, all wrapped in a cache-aware validation and submission pipeline.

13 Common misunderstandings corrected

(Misunderstanding 1)

"TF-IDF finds repeated words."
Only partly. It weights within-document frequency by inverse document frequency across the corpus. Repetition matters, but rarity matters too.

(Misunderstanding 2)

"BM25 removes stopwords."
Not inherently. BM25 is a ranking formula. Stopword removal is a preprocessing design choice. In this notebook, stopword filtering is enabled by configuration for lexical features; it is not an intrinsic property of BM25 itself.

(Misunderstanding 3)

"Embeddings are just another kind of token count."
No. Dense embeddings are learned vector representations intended to capture semantic proximity, including paraphrase-level similarity.

(Misunderstanding 4)

"Low precision means the system failed."
Not necessarily. With very large K, precision can be numerically tiny while recall and MRR remain strong. Metric interpretation depends on task design.

(Misunderstanding 5)

"A cross-encoder is just a slower embedding model."
Not exactly. A bi-encoder embeds query and document separately, while a cross-encoder scores the query-document pair jointly. The extra cost comes from this more expressive pairwise interaction.

(Misunderstanding 6)

"If the offline score is higher on the train queries, the system is better."
Not necessarily. If the same labeled queries were used for fitting and evaluation, the score can be optimistic because of leakage. Validation protocol matters as much as the raw number.

14 How to explain this project academically in class or in a report

14.1 Short version

> "We implemented TF-IDF, BM25+, and dense embedding retrieval as first-stage methods. The final active system uses dense embeddings for candidate generation, then applies a cross-encoder reranker to the top candidates and adds a soft category-aware bonus. Hyperparameters are tuned on a stratified validation split to avoid leakage, and the final reranker is retrained on all labeled training queries before submission."

14.2 Longer version

"The project begins with strict schema validation and shared normalization to make model comparisons trustworthy. Documents are represented as title + body + tags, while queries use title + body or title + body + tags depending on the subtask. TF-IDF provides a sparse cosine baseline, BM25+ provides a stronger lexical baseline with term-frequency saturation and document-length normalization, and dense embeddings provide a semantic first-stage retriever robust to paraphrase. Because dense retrieval is computationally expensive, the notebook caches document embeddings and scores queries in chunks. A TF-IDF + LinearSVC classifier predicts query category, while a cross-encoder reranks the most promising candidates jointly at the pair level. To avoid optimistic offline scores, the notebook tunes `top_k`, `top_m`, and category bonus on a stratified validation split, then retrains the cross-encoder on all labeled training queries before generating the final submission."

15 A practical tuning checklist

1. Verify data loading and ids before changing models.
2. Keep normalization shared across methods.
3. Establish TF-IDF as the sanity-check baseline.
4. Compare BM25+ against TF-IDF to isolate lexical improvements.
5. Move to embeddings when semantic mismatch is the main failure mode.

6. Cache document embeddings immediately.
7. Split labeled train queries into stratified train and validation subsets before tuning the cross-encoder.
8. Tune `top_k` for recall first, then tune `top_m` and category bonus for top-rank quality.
9. Inspect whether the category bonus helps ordering or overpowers relevance.
10. Report Recall, Precision, MRR, and Accuracy together rather than in isolation.
11. Reuse max-K results for smaller K sweeps when possible.
12. Retrain on all labeled training queries only after hyperparameters are fixed.

A Formula sheet

How to use this appendix

Do not read each formula as a block of symbols. First identify the **output** on the left-hand side, then identify the **main ingredients** on the right-hand side, and finally translate it into one plain-English sentence.

TF-IDF

$$\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \text{idf}(t)$$

$$\text{idf}(t) = \log\left(\frac{N}{\text{df}(t)}\right)$$

- **Read it as:** the weight of a term is larger when it appears often in this document and in relatively few documents overall.
- $\text{tf}(t, d)$: frequency of term t inside document d.
- $\text{df}(t)$: number of documents that contain t.
- N: total number of documents.

Cosine similarity

$$\cos(\theta) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \, \|\mathbf{d}\|}$$

- **Read it as:** compare the query vector and the document vector by alignment, not just by raw size.
- The numerator rewards shared weighted directions.
- The denominator rescales by vector length.

BM25+

$$\text{score}(q, d) = \sum_{t \in q} \text{idf}(t) \left(\frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)} + \delta \right)$$

- **Read it as:** for each query term, reward rarity and in-document frequency, but with diminishing returns and length correction.
- $f(t, d)$: number of times t appears in d.
- $|d|$: document length.
- avgdl: average document length.
- k_1: saturation strength.
- b: length-normalization strength.
- δ: BM25+ offset.

Recall@K

$$\text{Recall@K} = \frac{|R_q \cap \hat{R}_{q,K}|}{|R_q|}$$

- **Read it as:** relevant documents found in the top-K, divided by all relevant documents that existed.
- Numerator: hits in the top-K.
- Denominator: total relevant items for this query.

Precision@K

$$\text{Precision@K} = \frac{|R_q \cap \hat{R}_{q,K}|}{|\hat{R}_{q,K}|}$$

- **Read it as:** relevant documents found in the top-K, divided by everything returned in the top-K.
- If you literally returned K items, then $|\hat{R}_{q,K}| = K$.

MRR

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{r_q}$$

- **Read it as:** average of the reciprocal of the first relevant rank.
- $r_q = 1$ gives a contribution of 1.
- $r_q = 5$ gives a contribution of 0.2.

Combined score

$$\text{Score} = \frac{1}{4}(\text{Recall} + \text{Precision} + \text{MRR} + \text{Accuracy})$$

- **Read it as:** the final evaluation gives equal weight to retrieval coverage, retrieval cleanliness, ranking quality, and category prediction quality.

B Glossary from zero

(Corpus)

The full document collection the retriever searches over. In this project the corpus contains 216,041 technical forum documents.

(Query)

The input problem statement or question for which the system must return relevant documents.

(Ground truth / qrels)

The labeled relevance information telling us which documents are relevant for which train queries.

(First-stage retriever)

The fast model that scans the whole corpus and returns a candidate set. Here that role is played by TF-IDF, BM25+, or the embedding retriever.

(Candidate set)

The shortlist of documents kept after first-stage retrieval. In the notebook this is controlled by `top_k`.

(Reranker)

A second-stage model that only inspects the candidate set and tries to improve local ordering at the top.

(Bi-encoder)

A model that embeds the query and document separately and compares those vectors later. This is efficient for large-scale retrieval.

(Cross-encoder)

A model that reads the query-document pair jointly and produces one relevance score for that pair.

(Hard negative)

A non-relevant document that still looks plausible, often because the first-stage retriever ranked it highly.

(Hyperparameter)

A setting chosen by the experimenter, such as `top_k`, `top_m`, or category bonus.

(Validation split)

The held-out labeled subset used to compare settings honestly during development.

(Leakage)

Any situation where information from evaluation data influences training or tuning and makes the measured score too optimistic.

(Stratified split)

A split that preserves class proportions approximately, so each category remains represented in train and validation.

(Category bonus)

A soft additive score that rewards query-document category agreement during reranking.

(Cache)

Saved artifacts such as embeddings, vectorizers, or trained models that can be reused instead of recomputed.

C A miniature end-to-end worked example

Why this appendix matters

Many students understand each piece in isolation but still do not "see the movie" of the whole pipeline. This appendix gives a miniature notebook-aligned example with a tiny corpus so the stages can be followed from start to finish.

Step 1: tiny data

Imagine we have four documents:

- d_1: `androidappcrashesonstartup`
- d_2: `androidappfreezeswhenloadingimages`
- d_3: `latexbibliographyerrorafterupdate`
- d_4: `pythonscriptslowonlargefiles`

And two labeled train queries:

- q_1: `myandroidapplicationcrasheswhenopening`
- q_2: `latexreferencesstoppedcompiling`

Suppose the ground truth says:

$$q_1 \to \{d_1\}, \qquad q_2 \to \{d_3\}.$$

Step 2: first-stage retrieval intuition

The embedding retriever might return, for q_1,

$$d_2,\ d_1,\ d_4,\ d_3.$$

This is not crazy. Document d_2 is semantically similar to q_1 because it is also about Android app failure behavior, even though it is not the true labeled target for this toy example.

Step 3: why reranking helps

Now suppose the cross-encoder assigns these pairwise scores:

Document	Embedding order	Cross-encoder score
d_2	1	1.40
d_1	2	2.20
d_4	3	0.15
d_3	4	0.02

The reranker now promotes d_1 above d_2, which improves MRR because the first relevant document moves earlier.

Step 4: add a category signal

Suppose the classifier predicts the category of q_1 as `android`. If d_1 and d_2 are also `android`, they might each receive a bonus of $+0.5$. If d_4 is `programmers` and d_3 is `tex`, they receive no bonus.

The final scores become:

$$d_1 : 2.70, \quad d_2 : 1.90, \quad d_4 : 0.15, \quad d_3 : 0.02.$$

This bonus does not change the whole logic of relevance. It only reinforces a category-consistent ordering.

Step 5: split train and validation honestly

In a real notebook, we do not want to train the cross-encoder on a query and then claim that the same query proves generalization. So we split the labeled queries:

- train split: used to fit the cross-encoder,
- validation split: used to tune `top_k`, `top_m`, and category bonus.

With only two toy queries, we would not get a meaningful validation split. But with 327 labeled queries, this becomes possible and important.

Step 6: a simplified notebook-style code example

Listing 4: A simplified study version of the notebook pipeline

```
# 1. Prepare the split
ce_train_query_ids, val_query_ids = stratified_query_train_validation_split(
    train_queries_df,
    validation_fraction=0.2,
    seed=42,
)

ce_train_queries_df = select_rows_by_ids(train_queries_df, ce_train_query_ids)
val_queries_df = select_rows_by_ids(train_queries_df, val_query_ids)

ce_train_ground_truth = subset_ground_truth(ground_truth, ce_train_query_ids)
val_ground_truth = subset_ground_truth(ground_truth, val_query_ids)

# 2. Train the reranker only on the train split
validation_cross_encoder = build_or_load_cross_encoder(
```

```
    train_queries_frame=ce_train_queries_df,
    docs_frame=docs_df,
    ground_truth=ce_train_ground_truth,
)

# 3. First-stage retrieval on validation queries
base_results = run_retrieval(
    model_name="embedding",
    docs_frame=docs_df,
    queries_frame=val_queries_df,
    top_k=max(TUNING_TOP_KS),
)

# 4. Sweep hyperparameters on validation only
for top_k in TUNING_TOP_KS:
    for top_m in TUNING_RERANK_TOP_MS:
        for bonus in TUNING_CATEGORY_BONUSES:
            reranked = rerank_results_with_cross_encoder(
                results=base_results,
                query_frame=val_queries_df,
                docs_frame=docs_df,
                cross_encoder=validation_cross_encoder,
                query_category_map=val_query_category_map,
                doc_category_map=doc_category_map,
                rerank_top_m=top_m,
                category_bonus=bonus,
            )
            metrics = leaderboard_score(
                truncate_results(reranked, top_k),
                val_ground_truth,
                k=top_k,
                accuracy_value=validation_classifier_accuracy,
            )

# 5. Retrain on all labeled train queries only after choosing the best setting
final_cross_encoder = build_or_load_cross_encoder(
    train_queries_frame=train_queries_df,
    docs_frame=docs_df,
    ground_truth=ground_truth,
)

# 6. Generate the final test submission
test_results = run_retrieval(... queries_frame=test_queries_df, top_k=best_top_k)
test_results = rerank_results_with_cross_encoder(... cross_encoder=final_cross_encoder)
write_kaggle_submission(test_results, ...)
```

How to read that code without panic

If the code looks long, reduce it to six questions:

1. Where do we split train and validation?
2. Which subset is used to fit the cross-encoder?
3. Which subset is used to compare hyperparameters?
4. Where is the first-stage retriever called?

5. Where is the cross-encoder reranker called?
6. At what point do we retrain on all labeled train queries?

If you can answer those six questions, then you understand the most important structure of the notebook.

D Reading the cross-encoder notebook code from zero

Why this section exists

Many students hear "cross-encoder" and imagine that a mysterious neural block is dropped into the notebook without explanation. The goal of this section is to slow the story down. We will connect the notebook functions to the exact learning problem they solve, show what one training pair looks like, and explain why the reranker can improve ranking even though it is trained with simple binary labels.

Question 1: what is the cross-encoder actually learning?

The cross-encoder is **not** learning whole-corpus retrieval directly. It is learning a smaller and more local task:

> Given one query and one candidate document, should this pair receive a high relevance score or a low relevance score?

That local decision is enough to improve ranking because the notebook uses the cross-encoder only on a shortlist. Once the first-stage retriever has already proposed plausible candidates, the reranker only has to answer:

> Among these few candidates, which pair looks most relevant?

This is why the cross-encoder can be trained with binary labels and still be useful for ranking. A ranking is just an ordering by score. If relevant pairs tend to get higher scores than non-relevant pairs, the ordering improves.

Question 2: what does one training example look like?

Each training example is a pair plus a label:

- positive pair: query "android app crashes on startup" with document "my application crashes immediately after launch on android 14" and label 1,
- negative pair: query "android app crashes on startup" with document "latex bibliography not showing with biblatex" and label 0,
- positive pair: query "latex references stopped compiling" with document "biblatex error after package update" and label 1,
- negative pair: query "latex references stopped compiling" with document "python loop over large files is slow" and label 0.

So when you see notebook code that builds `InputExample(texts=[query_text,doc_text],label=1.0)` or label 0.0, do not let the object name scare you. It is just packaging one supervised example for the model.

Question 3: what does the model "see" as input?

At the conceptual level, the model sees *both texts together*. In many transformer implementations, this means something close to:

Conceptual cross-encoder input

```
[CLS] query tokens [SEP] document tokens [SEP]
```

You do not need to memorize tokenizer details to understand the pipeline. The important idea is simpler:

- a bi-encoder encodes each side separately,
- a cross-encoder creates one joint representation for the whole pair.

That joint view lets the model notice interactions such as:

- query word `startup` aligns with document phrase `afterlaunch`,
- query phrase `referencesstoppedcompiling` aligns with document phrase `biblatexerrorafterupdate`,
- one candidate uses the right technical vocabulary but the wrong category and context.

Question 4: what score comes out of the cross-encoder?

The model outputs one scalar for the pair. For teaching purposes, you can think of the flow as:

$$\text{pair text} \longrightarrow f_\theta(q, d) \longrightarrow \hat{y}(q, d),$$

where $f_\theta(q, d)$ is a raw neural score and $\hat{y}(q, d)$ is the interpreted relevance signal.

Sometimes that score is treated like a logit, sometimes like a direct regression-style score, depending on the library setup. For the notebook-level understanding, the exact implementation detail is less important than the ordering rule:

> larger score = this query-document pair looks more relevant.

Question 5: why build both easy and hard negatives?

If every negative example were obviously wrong, the reranker would learn too little. It would become good at rejecting absurd mismatches, but not good at separating plausible near-misses.

That is why the notebook mixes:

- **easy or random negatives:** clearly unrelated documents,
- **hard negatives:** documents that looked good to the first-stage retriever but are still not relevant.

A teaching analogy

If you want to learn to distinguish identical twins, training only on photos of twins versus photos of bicycles will not help much. You also need difficult near-cases. Hard negatives are those near-cases for retrieval.

Question 6: how are hard negatives mined in plain English?

The notebook first uses a retrieval model to get a high-ranking candidate list for each labeled query. Then it removes the truly relevant documents and keeps some of the remaining top-ranked wrong documents as hard negatives.

The logic looks like this:

1. take one labeled query,
2. run the fast retriever,
3. collect top-ranked documents,
4. remove the documents listed in ground truth,
5. keep some of the surviving documents as difficult wrong examples.

Here is a tiny illustration:

Rank from fast retriever	Document	Relevant?	Use as hard negative?
1	d_{17}	yes	no
2	d_{204}	no	yes
3	d_{88}	no	yes
4	d_{310}	yes	no
5	d_{415}	no	maybe

This table is important because it shows that hard negatives are not random from the whole corpus. They are selected from *the retriever's own mistakes.*

Question 7: what does the pair-building code mean line by line?

When the notebook calls helper functions such as `build_text_map`, `mine_hard_negative_doc_ids`, and `build_cross_encoder_training_examples`, here is the conceptual translation:

- `build_text_map(...)`: create a lookup table from ids to the final text string that will be fed to models.
- `mine_hard_negative_doc_ids(...)`: ask the first-stage retriever for plausible but wrong documents.
- `build_cross_encoder_training_examples(...)`: turn positive and negative pairs into supervised examples.
- `DataLoader(...)`: group many examples into batches for efficient training.
- `cross_encoder.fit(...)`: update the neural weights so positive pairs tend to score above negative pairs.

Question 8: what is the loss function doing without too much math?

During training, the model compares its current prediction to the known label:

- if a positive pair gets a low score, the loss says "increase this",
- if a negative pair gets a high score, the loss says "decrease this".

After many batches, the model gradually reshapes its parameters so the score surface separates relevant and non-relevant pairs better.

If you want one compact formula, a common mental model is binary cross-entropy:

$$\mathcal{L}(q, d, y) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y}),$$

where $y \in \{0, 1\}$. But the main thing to understand is not the formula itself. The main thing is:

> training repeatedly punishes the model when it gives the wrong relative signal to a pair.

Question 9: why can a binary pair model improve a ranked list?

Suppose the first-stage retriever returns five candidates. The cross-encoder scores them like this:

$$2.2,\ 1.7,\ 0.9,\ 0.2,\ -0.1$$

Those numbers do not need to be probabilities. They only need to be *comparable*. Sorting by them gives a better local ranking if relevant candidates usually rise above non-relevant ones.

So the reranking rule is extremely simple:

> take the candidate list, rescore candidate pairs, and sort again by the new score.

Question 10: a notebook-style code reading example

Listing 5: Study version of the cross-encoder training block

```
query_text_map = build_text_map(ce_train_queries_df)
doc_text_map = build_text_map(docs_df)

hard_negative_doc_ids_by_query = mine_hard_negative_doc_ids(
    train_queries_frame=ce_train_queries_df,
    docs_frame=docs_df,
    ground_truth=ce_train_ground_truth,
    query_ids=ce_train_query_ids,
    top_k=CROSS_ENCODER_HARD_NEGATIVE_TOP_K,
)

training_examples = build_cross_encoder_training_examples(
    query_text_map=query_text_map,
    doc_text_map=doc_text_map,
    ground_truth=ce_train_ground_truth,
    query_ids=ce_train_query_ids,
    max_positives_per_query=4,
    negatives_per_positive=4,
    hard_negative_doc_ids_by_query=hard_negative_doc_ids_by_query,
)

train_loader = DataLoader(training_examples, shuffle=True, batch_size=16)
cross_encoder.fit(train_dataloader=train_loader, epochs=5)
```

Read it in slow motion:

1. Convert tables into text lookups.
2. Mine difficult wrong answers for each training query.
3. Build labeled pairs.
4. Put those pairs into batches.
5. Train for several passes through the data.

Question 11: what should you remember most?

If the variable names still feel intimidating, reduce the whole cross-encoder block to one sentence:

> The notebook collects good and bad query-document pairs, then trains a joint pair-scoring model so that relevant candidates rise to the top of a shortlist.

E A worked validation and tuning simulation

Why this section matters

Validation is often described in one sentence, but the real difficulty is understanding what changes across experiments and what must stay fixed. This section gives a slow notebook-style simulation of model selection so you can see how `top_k`, `top_m`, and category bonus are tuned without leakage.

Step 1: create one honest split

Imagine the 327 labeled queries are divided into:

- 261 cross-encoder training queries,
- 66 validation queries.

The exact counts can vary slightly depending on the split function, but the core rule stays the same:

> training queries are for fitting, validation queries are for choosing settings.

Step 2: train only on the cross-encoder train fold

The reranker is fit using only the 261 training queries and their relevant documents. The 66 validation queries are untouched during fitting.

At this point, if you are reading the notebook, you should ask yourself one simple safety question:

> Did the model see these validation query ids during training pair construction?

If the answer is no, the protocol is still honest.

Step 3: run the expensive first stage only once at the largest needed depth

Suppose you want to compare:

$$\texttt{top\_k} \in \{1000, 3000, 5000\}.$$

Instead of rerunning retrieval three times separately, the notebook can retrieve once at:

$$\max(\texttt{top\_k}) = 5000$$

and then truncate the result list to 1000 or 3000 when needed.

This is both efficient and conceptually clean:

- retrieve once,
- reuse the same base ranking,
- compare settings fairly.

Step 4: sweep the reranking depth and category bonus

Now suppose we also try:

$$\texttt{top\_m} \in \{20, 50, 100\}, \qquad \texttt{categorybonus} \in \{0.0, 0.2, 0.5\}.$$

Then the validation loop is exploring combinations such as:

- retrieve 5000, rerank 20, no bonus,
- retrieve 5000, rerank 50, bonus 0.2,
- retrieve 3000, rerank 100, bonus 0.5.

Each combination defines one candidate system.

Step 5: read a tuning table like a researcher

Here is a fully hypothetical validation table:

top_k	top_m	bonus	Avg. score	Interpretation
1000	20	0.0	0.281	shortlist is small; reranking light
1000	50	0.2	0.289	reranking helps a bit
3000	20	0.2	0.301	more recall from first stage
3000	50	0.2	0.309	stronger balance
5000	50	0.2	0.311	small gain from deeper retrieval
5000	100	0.5	0.307	too much extra reranking/bonus

What should a student learn from such a table?

- increasing `top_k` can help recall,
- increasing `top_m` can help ordering,
- too much category bonus can start to distort relevance,
- the best setting is not the one with the biggest component value but the one with the best validation score.

Step 6: why the best setting is chosen on validation only

Suppose the best row is:

$$\texttt{top\_k} = 5000, \quad \texttt{top\_m} = 50, \quad \text{bonus} = 0.2.$$

That decision must come from validation, not from the final test set and not from the full 327 queries after the reranker has already seen them all.

This is exactly what validation is protecting:

> it separates *system design* from *final test-time use*.

Step 7: what the notebook stores as `best\_experiment`

When you see a variable like `best_experiment`, read it as:

> the best-performing hyperparameter configuration on the held-out validation queries.

It is usually a dictionary or record containing things like:

- the retrieval model name,
- chosen `top_k`,
- chosen `top_m`,
- chosen category bonus,
- metric values measured on validation.

So `best_experiment` is not "the final Kaggle score." It is the notebook's memory of which development setting won the honest comparison.

Step 8: why retraining after tuning is not leakage

Students often worry here: if we retrain on all 327 queries after tuning, did we break the protocol again?

The answer is:

- **No, for submission generation:** this is normal and correct.
- **Yes, if you then report a new offline score on those same 327 queries as if it were fair.**

So the rule is:

> retrain on all labeled queries to produce the strongest final submission, but keep the reported development comparison on the validation split.

Step 9: a notebook-style validation loop

Listing 6: Study version of a leakage-safe tuning loop

```
base_results = run_retrieval(
    model_name="embedding",
    docs_frame=docs_df,
    queries_frame=val_queries_df,
    top_k=max(TUNING_TOP_KS),
)

experiments = []
for top_k in TUNING_TOP_KS:
    truncated_results = truncate_results(base_results, top_k)
    for top_m in TUNING_RERANK_TOP_MS:
        for bonus in TUNING_CATEGORY_BONUSES:
            reranked_results = rerank_results_with_cross_encoder(
                results=truncated_results,
                query_frame=val_queries_df,
                docs_frame=docs_df,
                cross_encoder=validation_cross_encoder,
                rerank_top_m=top_m,
                category_bonus=bonus,
            )
            metrics = leaderboard_score(
                reranked_results,
                val_ground_truth,
```

```
                k=top_k,
                accuracy_value=validation_classifier_accuracy,
            )
            experiments.append({
                "top_k": top_k,
                "top_m": top_m,
                "bonus": bonus,
                "avg_score": metrics["score"],
            })

best_experiment = max(experiments, key=lambda row: row["avg_score"])
```

Step 10: how to read that loop without panic

There are only four ideas inside this code:

1. get one base candidate ranking on validation queries,
2. try several truncation depths,
3. try several reranking and bonus settings,
4. keep the configuration with the best validation score.

Step 11: what mistakes this protocol prevents

This protocol protects against several common notebook mistakes:

- training and evaluating on the same labeled queries,
- selecting `top_m` by repeatedly inspecting the same training examples,
- claiming a final post-retraining train score as if it were an honest development estimate,
- comparing systems that were not evaluated on the same held-out subset.

Step 12: the one-sentence summary

If you need one sentence for memory, use this:

> Validation is the notebook's protected decision zone: all hyperparameter choices are made there before the final reranker is retrained on all labeled data for submission.

F Notebook variable names translated into plain English

Why students need this

Many notebook readers are blocked not by machine learning itself but by naming density. DataFrame names, cached objects, score maps, and helper outputs can look overwhelming. This section translates the most important variable names into ordinary language.

The core dataset objects

- `docs_df`: the documents table, which is the big searchable corpus.
- `train_queries_df`: the labeled queries table, which contains the 327 training queries with known relevant documents.
- `test_queries_df`: the unlabeled query table, which contains the Kaggle queries for which the notebook must predict results.
- `ground_truth`: the relevance mapping, which acts as the answer key for train queries only.
- `doc_texts` or text map: the final document strings built from title, body, and tags.
- `query_texts` or text map: the final query strings after shared preprocessing.
- `doc_category_map`: the mapping from document id to its known forum category.

The split-related objects

- `ce_train_query_ids`: the training query ids that are allowed to teach the cross-encoder.
- `val_query_ids`: the validation query ids reserved for honest system comparison.
- `ce_train_queries_df`: the train slice of the 327 labeled queries.
- `val_queries_df`: the held-out slice used for tuning.
- `ce_train_ground_truth`: the relevance labels for the reranker's training fold only.
- `val_ground_truth`: the relevance labels for the tuning fold only.

The retrieval and reranking objects

- `embedding_model`: the sentence-transformer retriever used as the fast semantic candidate generator.
- `cross_encoder`: the slower pair-scoring reranker that judges query-document pairs jointly.
- `validation_cross_encoder`: the fold-specific reranker trained only on the train split for honest tuning.
- `final_cross_encoder`: the reranker retrained on all 327 labeled queries after tuning.
- `base_results`: the candidate lists returned before reranking.
- `reranked_results`: the same candidate lists after pairwise rescoring and sorting.
- `hard_negative_doc_ids_by_query`: wrong documents that still looked plausible for each train query.
- `training_examples`: the positive and negative query-document examples used to fit the reranker.

The category-related objects

- `category_vectorizer`: the TF-IDF feature builder for the classifier.
- `category_classifier`: the `LinearSVC` model that predicts forum category from query text.
- `train_query_category_map`: the category labels attached to training queries.
- `val_query_category_map`: the category labels used while tuning on the held-out fold.
- `predicted_test_categories`: the classifier's guesses for the unlabeled Kaggle queries.

The tuning and experiment objects

- `TUNING_TOP_KS`: the candidate values of `top_k` the notebook will compare.
- `TUNING_RERANK_TOP_MS`: the candidate values of `top_m` the notebook will compare.
- `TUNING_CATEGORY_BONUSES`: the candidate category bonus strengths the notebook will compare.
- `experiments`: the notebook's record of every configuration tested on validation.
- `best_experiment`: the best-performing row from that experiment table.
- `metrics`: the evaluation outputs such as Recall, Precision, MRR, Accuracy, and the combined score.

How to read a notebook cell when you feel lost

When you open a dense code cell, do not try to understand every line at once. Use this four-question method:

1. Which objects are inputs?
2. Which new object is being created?
3. Is this a training step, an evaluation step, or a cache-loading step?
4. Which conceptual stage of the pipeline does this belong to?

For example, if you see:

Listing 7: A tiny code-reading exercise

```
reranked_results = rerank_results_with_cross_encoder(
    results=base_results,
    query_frame=val_queries_df,
    docs_frame=docs_df,
    cross_encoder=validation_cross_encoder,
    rerank_top_m=50,
    category_bonus=0.2,
)
```

you can translate it into plain English as:

> Take the validation candidate lists, let the validation-trained reranker rescore only the top 50 documents per query, add a small category bonus where appropriate, and return the new sorted rankings.

How to classify notebook cells by role

Almost every important cell in the notebook falls into one of these roles:

- **data preparation:** load tables, check ids, build text,
- **representation building:** vectorizer fitting or embedding computation,
- **model fitting:** classifier or cross-encoder training,
- **retrieval execution:** generate candidate rankings,
- **reranking execution:** rescore top candidates,
- **evaluation and tuning:** compute metrics and compare settings,

- **final submission:** retrain on all labels and write the Kaggle file.

If you can assign each code cell to one of these roles, the notebook stops looking like random Python and starts looking like a system.

A compact translation exercise

Here is a tiny block:

Listing 8: Translate the code into the pipeline story

```
ce_train_queries_df = select_rows_by_ids(train_queries_df, ce_train_query_ids)
val_queries_df = select_rows_by_ids(train_queries_df, val_query_ids)

validation_cross_encoder = build_or_load_cross_encoder(
    train_queries_frame=ce_train_queries_df,
    docs_frame=docs_df,
    ground_truth=ce_train_ground_truth,
)
```

The plain-English translation is:

1. split the labeled training queries into a fitting subset and a held-out subset,
2. train the reranker only on the fitting subset,
3. keep the held-out subset clean for honest tuning.

The one rule that simplifies almost everything

Whenever a variable name looks abstract, ask:

> Is this variable storing data, a model, a score list, or a chosen setting?

That one question is enough to decode a surprising amount of notebook structure.

G One query's journey through the full notebook pipeline

Why this final appendix helps

The best way to understand a retrieval system is often to follow one query from beginning to end. This section tells the whole story of one query as it moves through preprocessing, first-stage retrieval, reranking, category adjustment, and metric evaluation.

Stage 1: raw query text enters the notebook

Suppose the user query is:

> `androidapplicationcrashesimmediatelyafterstartup`

At this moment, the notebook has not done retrieval yet. It first needs to build the same kind of cleaned text representation used throughout the project.

Stage 2: preprocessing and text construction

The query is normalized using the shared preprocessing policy. Depending on the notebook configuration, this may include:

- lowercasing,
- punctuation normalization,
- whitespace cleanup,
- shared title/body formatting.

The important conceptual rule is:

> the query should be processed in a way that is compatible with how document texts were built.

Stage 3: optional category prediction

The category classifier reads the query text and predicts a forum domain, for example:

$$\widehat{\text{category}}(q) = \texttt{android}.$$

Nothing is filtered yet. The prediction is simply stored because later it may add a soft bonus to category-consistent documents.

Stage 4: dense first-stage retrieval

The embedding model converts the query into a vector $e(q)$. Each document already has a stored vector $e(d)$ from a previous encoding step. The notebook computes similarities such as:

$$\text{sim}(q, d) = \cos(e(q), e(d)).$$

Imagine the top results after this fast stage are:

Document	Category	Dense similarity	Kept in top-K?
d_1	android	0.88	yes
d_2	programmers	0.85	yes
d_3	android	0.83	yes
d_4	unix	0.79	yes
d_5	tex	0.76	yes

The notebook may keep thousands of documents at this stage, but the table above is enough to understand the flow.

Stage 5: select the top-m for reranking

From the larger top-K pool, the notebook keeps only the top-m documents for expensive pairwise rescoring. If `top_m=3`, then only:

$$d_1,\ d_2,\ d_3$$

are passed to the cross-encoder.

This stage is crucial because it controls computational budget. The system is saying:

> I will inspect only the most promising candidates carefully.

Stage 6: build query-document pair texts

For each selected candidate, the notebook forms a pair:

- pair 1: (q, d_1),
- pair 2: (q, d_2),
- pair 3: (q, d_3).

Conceptually, the model now receives three separate questions:

- how relevant is d_1 to this exact query?
- how relevant is d_2 to this exact query?
- how relevant is d_3 to this exact query?

Stage 7: cross-encoder rescoring

Suppose the cross-encoder outputs:

$$\text{cross}(q, d_1) = 1.9, \quad \text{cross}(q, d_2) = 1.1, \quad \text{cross}(q, d_3) = 2.3.$$

Now the local order changes. Before reranking, d_1 was above d_3. After reranking, d_3 moves above d_1.

This is the whole point of the second stage:

> the cross-encoder corrects local ordering mistakes made by the faster first-stage retriever.

Stage 8: apply the category bonus

Suppose the predicted query category is `android`, and d_1 and d_3 are also `android`, while d_2 is `programmers`. If the category bonus is 0.2, then the final scores become:

$$d_1 : 1.9 + 0.2 = 2.1,$$

$$d_2 : 1.1 + 0.0 = 1.1,$$

$$d_3 : 2.3 + 0.2 = 2.5.$$

The final order is:

$$d_3 \succ d_1 \succ d_2.$$

Stage 9: merge reranked and untouched candidates

Only the reranked prefix changes. The rest of the larger top-K list stays behind it in the original order unless the notebook explicitly rebuilds the whole list around the reranked prefix.

So the returned ranking is often conceptually:

- reranked top-m,
- then the remaining candidates from positions $m + 1$ to K.

Stage 10: compare the final ranking to ground truth

If this is a validation query, the notebook can compare the ranked results to the known relevant document set. Suppose the true relevant set is:

$$R_q = \{d_3, d_7\}.$$

If the final top-5 list starts with:

$$d_3,\ d_1,\ d_2,\ d_{11},\ d_7,$$

then:

- Recall@5 is $2/2 = 1.0$ because both relevant documents were recovered,
- Precision@5 is $2/5 = 0.4$ because two of the five are relevant,
- reciprocal rank is $1/1 = 1.0$ because the first relevant document is at rank 1.

Stage 11: where validation matters in this story

If this query belonged to the cross-encoder training fold, then using it to judge the tuned system would be optimistic. If it belongs to the validation fold instead, then it is doing the right job:

> it is helping us estimate how well the tuned system behaves on unseen labeled queries.

Stage 12: where the Kaggle submission differs

For test queries, the story is almost the same except for one key difference:

- we do not have ground truth labels,
- so the notebook cannot compute Recall, Precision, or MRR,
- it can only output the final ranked document ids.

That is why the test set is for submission, not for development decisions.

A complete story in compact pseudocode

Listing 9: One query moving through the pipeline

```
query_text = normalize_text(raw_query)
predicted_query_category = category_classifier.predict([query_text])[0]

candidate_results = embedding_retrieve(
    query_text=query_text,
    docs_frame=docs_df,
    top_k=best_top_k,
)

reranked_results = rerank_results_with_cross_encoder(
    results=candidate_results,
    query_frame=single_query_df,
    docs_frame=docs_df,
    cross_encoder=final_cross_encoder,
    rerank_top_m=best_top_m,
    category_bonus=best_category_bonus,
    query_category_map={query_id: predicted_query_category},
```

```
    doc_category_map=doc_category_map,
)

top_doc_ids = extract_doc_ids(reranked_results)
```

What to remember from the journey

If you remember only one image, remember this:

> one query first enters a fast semantic search stage, then a slower pairwise judgment stage, then a small category-aware adjustment stage, and only after that becomes a final ranked answer list.

H Fifteen oral-exam questions with model answers

1. **Why is TF-IDF sparse?** Because each vector has one dimension per vocabulary item, and most terms are absent from a given document.
2. **Why is BM25+ usually better than raw TF-IDF for lexical retrieval?** Because it includes term-frequency saturation and document-length normalization, which better reflect retrieval behavior on uneven documents.
3. **Why are embeddings useful here?** Because relevant technical documents may use different wording from the query, and dense embeddings are more robust to paraphrase.
4. **Why not submit TF-IDF if it is simpler?** Because a simple baseline is valuable for comparison, but the final choice should follow empirical performance on the task.
5. **Why use a category classifier?** To estimate the topical region of the query and reduce cross-category noise in deep rankings.
6. **What is the difference between a bi-encoder and a cross-encoder?** A bi-encoder embeds query and document separately for fast large-scale retrieval, while a cross-encoder scores the pair jointly for more accurate reranking.
7. **Why not run the cross-encoder on the whole corpus?** Because pairwise joint scoring is much slower than vector similarity search, so it is practical only on a shortlist of candidates.
8. **What does `top_m` mean?** It is the number of top candidates from first-stage retrieval that are actually reranked by the cross-encoder.
9. **What is a hard negative?** A document that is not relevant but looks plausible, often because the first-stage retriever ranked it highly.
10. **Why is a validation split necessary here?** Because tuning or evaluating on the same labeled queries used for cross-encoder fitting would produce optimistic scores due to leakage.
11. **Why stratify the split by category?** To keep category proportions similar in train and validation so the validation score remains representative across the five forum domains.
12. **Why retrain on all 327 labeled queries after tuning?** Once the best hyperparameters are fixed, using all available training labels gives the final submission model the strongest possible training signal.
13. **Why is precision small when K is large?** Because the denominator includes many returned documents; even strong recall can coexist with low precision at large K.
14. **Why cache embeddings and trained models?** Because document encoding and reranker loading are expensive and do not need to be repeated every run.

15. **Why use query chunking and partial sorting?** Query chunking controls memory usage during dense scoring, and partial sorting avoids an unnecessary full sort when only the top part of the ranking is needed.

I Final takeaway

What you should remember most

This project is not just "TF-IDF versus BM25 versus embeddings." It is a full retrieval system with an honest development protocol. The final notebook combines semantic first-stage retrieval, cross-encoder reranking, lightweight category-aware score adjustment, stratified validation-based tuning, final retraining on all labeled training queries, and strong runtime engineering. The best academic description is: *a cache-aware two-stage retrieval pipeline with dense candidate generation, pairwise reranking, lightweight category priors, and leakage-safe validation.*